



Iota

Platform Handbook & Administrator Guide

An online Mathematics examination platform for JEE aspirants. A teacher writes questions in LaTeX and publishes timed papers; a student sits a 25-question, 60-minute examination with a server-authoritative clock and autosaved answers, then receives a scored, ranked result with question-by-question analytics.

Angular 22

Spring Boot 3.5

Java 21 LTS

PostgreSQL 17

KaTeX

Chart.js

Flyway

JWT

Modular monolith

Document

Platform Handbook — v1.5

Applies to

Iota 1.0.0

Compiled

30 August 2026

Audience

Administrators, teachers, developers

Classification

Proprietary — internal use

Contents

1	About the platform	16	Ranking, percentile and metrics
2	Roles and what each can do	17	The student journey
3	Feature catalogue	18	Authentication and security
4	Technology stack	19	API reference
5	System architecture	20	Error handling contract
6	Project structure	21	Installation runbook
7	Data model	22	Environment variables
8	Adding a question — walkthrough	23	Database and migrations
9	Question lifecycle	24	Seed data
10	Writing questions in LaTeX	25	Deployment
11	Subjects and chapters	26	Testing
12	Marking schemes	27	Architecture decisions
13	Creating and publishing tests	28	Limitations and roadmap
14	Students and analytics	29	Glossary
15	The marking engine		

How to use this handbook

Sections 1–7 describe what the platform is and how it is put together. Sections 8–14 are the administrator operating manual — section 8 in particular is the complete, step-by-step procedure for adding a question, and is the section most administrators will return to. Sections 15–20 cover the engine and the contracts a developer works against. Sections 21–26 are the runbook for standing the system up, and 27–29 record the reasoning behind the design.

SOURCE OF TRUTH

This handbook was compiled from the project repository itself — schema migrations, entity and DTO definitions, validators, controllers, the Angular admin components, and the project documentation set (`README.md` , `docs/API.md` , `docs/ARCHITECTURE.md` , `docs/DEPLOYMENT.md`). Where behaviour is enforced in code, the enforcing rule is quoted rather than paraphrased.

1 About the platform

Iota is a single-subject, high-stakes examination platform built around one idea: a result, once produced, must never move.

A teacher writes mathematics questions as LaTeX source, classifies them, and publishes them into a question bank. From that bank they compose timed tests. A student registers with a phone number, browses the published papers, and sits one: 25 questions, 60 minutes, one question at a time, with palette navigation, mark-for-review, and answers saved as they go. When the paper is submitted — by the student, or automatically when the clock expires — the server scores it from an immutable snapshot, ranks the student against everyone who sat the same paper, and releases the answer key and worked solutions for review.

1.1 The name

The platform is called **iota**. The name is a double reference, and both halves are apt for a JEE Mathematics product. In Indian mathematics teaching the imaginary unit $i = \sqrt{-1}$ is routinely read aloud as "iota"; and ι is the Greek letter iota, proverbially the smallest quantity — the mark at the margin.

The mark on the cover reads at three levels. The **dot above a curved stem** is the letter ι itself. The **faint tick to its right is a radical sign** — $\sqrt{-1}$, the quantity "iota" names — carried at 50% opacity on purpose, so it reads as texture at large sizes and disappears cleanly at small ones. Below roughly 24 pixels the radical is dropped entirely and the dot and stem thicken; that reduction is the favicon. The brand gradient runs indigo #6366f1 through #4338ca to violet #7c3aed, and this handbook is set in the same indigo family.

THE CODE STILL SAYS MATHSTROKES — THIS IS CORRECT, NOT STALE

Iota was previously called MathStrokes, and the **product** name has been changed everywhere it is user-facing. A number of **identifiers** deliberately have not been renamed, because renaming them would be a migration rather than a rebrand, and this handbook documents the system as it actually is. Expect to keep seeing the old name in:

- the Java package `com.mathstrokes.*` and `MathStrokesApplication`;
- the PostgreSQL database, role and JDBC URL — all `mathstrokes`;
- the repository `github.com/vivekkumarq/MathStrokes` and the working directory `MathStrokesTestPlatform`;
- the frontend CSS tokens `--ms-*` and the utility classes `.ms-btn` and `.ms-card`, which are shared across files and are not safe to rename piecemeal.

Every code sample, path and command in this handbook uses the real identifier, so it can be copied and run exactly as printed.

1.2 What makes it different

Property	How the platform guarantees it
Results are immutable	Every attempt copies the question text, options, answer key and marking configuration onto itself when it starts. Evaluation reads only that copy. A teacher who later rewrites a question, flips its key or archives it cannot move a mark already awarded.
The server owns the clock	<code>expires_at</code> is written once when the attempt starts and is never extended. The browser countdown displays the server arithmetic; it is not a source of truth.
The answer key cannot leak	The payload a live attempt returns has no correctness field and no solution text — it is a structurally different type from the review payload, so the key cannot escape through a forgotten filter.
Nothing is lost to a bad connection	Each autosave carries a monotonic sequence number. A write arriving out of order is a late packet: it is discarded rather than allowed to overwrite newer work, and the response returns the state the server actually holds.
Abandoning still gets marked	A scheduled sweep finalises attempts whose time has run out, scoring the answers already saved. Closing the tab does not void the paper.
Scoring is exact	All marks are <code>NUMERIC</code> / <code>BigDecimal</code> , never floating point, so partial-marking arithmetic is precise.

2 Roles and what each can do

There are exactly two roles. There is no role hierarchy and no third tier.

Role	Reaches	Responsible for
<code>ROLE_ADMIN</code> Teacher / administrator	<code>/api/admin/**</code> <code>/api/catalog/**</code> <code>/api/profile</code>	Subjects and chapters, the question bank, marking schemes, test creation and publication, the student roster, and analytics.
<code>ROLE_STUDENT</code> Examination candidate	<code>/api/tests</code> <code>/api/attempts/**</code> <code>/api/me/performance</code> <code>/api/rankings/**</code> <code>/api/catalog/**</code>	Browsing published tests, sitting papers, reviewing their own finished attempts, and tracking their own performance.

HOW THE BOUNDARY IS ENFORCED

Path level: `/api/admin/**` requires `ROLE_ADMIN` in `SecurityConfig`; student routes carry `@PreAuthorize`. Object level — and this is the real guard — **no endpoint accepts a student id**. The caller is always derived from the security context, and every attempt is loaded through a single method that proves ownership. A student requesting another student's attempt receives `403`.

REGISTRATION CANNOT ESCALATE

POST `/api/auth/student/register` always creates a student; the role is not a field on the request and cannot be chosen. Additional administrators are created by seeding, not through the public API.

3 Feature catalogue

3.1 Teacher / administrator

Feature	Detail
Catalogue management	Create and edit subjects and chapters; activate or deactivate a chapter. The student app reads these from the API rather than hardcoding them.
LaTeX question authoring	Write the stem, every option and the worked solution in LaTeX, with a live KaTeX preview of each as you type.
Three-axis classification	Exam pattern (JEE Main / JEE Advanced), difficulty (Easy / Medium / Hard) and question type (single correct / multiple correct) are independent axes.
Question lifecycle	Draft → Published → Archived, with revert-to-draft. No hard delete.
Bank search	Paginated grid filterable by subject, chapter, pattern, difficulty, type and status, plus free-text search over stem and solution.
Marking schemes	Configure marks per outcome, including JEE Advanced partial credit, without touching code.
Test composition	Create tests, set duration and question count, choose a difficulty blueprint, publish, close and archive.
Student roster	Search students; enable or disable an account.
Analytics	Headline dashboard counters, hardest questions by measured accuracy, and average score and accuracy by chapter.

3.2 Student

Feature	Detail
Registration	Full name, phone number, password and a security question with its answer. No email anywhere in the system.
Password recovery	Three-step security-question flow. Knowing a phone number alone yields nothing but the question.
Test browsing	Published tests filtered by chapter and exam pattern, each showing whether it can be started and whether one is already in flight.
The examination	25 questions over 60 minutes, one at a time, with palette navigation, mark-for-review, and autosave.
Crash resilience	Refresh, crash or close the tab and return to the same paper — same questions, same order, clock still running.
Results	Score, rank, percentile, accuracy, attempt rate and negative marks.
Answer review	Question-by-question review with the answer key and worked solutions, rendered as the paper was sat.
Performance tracking	Attempt history and performance over time and by chapter.

DELIBERATELY NOT PRESENT

The platform contains **no AI, OCR, handwriting recognition or LLM integration**; no file or object storage and **no image upload on questions** (mathematics is LaTeX text, so there are no diagrams to store); no cache, message queue, microservices or Kubernetes; no bulk question import endpoint; no email or SMS; and no OAuth, sessions or cookies — JWT bearer only. Each of these is a recorded decision, not an oversight.

4 Technology stack

4.1 At a glance

Layer	Choice	Notes
Frontend framework	Angular 22	Standalone components, zoneless change detection, signals, lazy-loaded routes
Frontend language	TypeScript 6	Strict; enums modelled as string-literal unions mirroring the backend
Styling	SCSS	Component-scoped, no CSS framework
Maths rendering	KaTeX	Renders stored LaTeX source at display time, HTML trust disabled
Charts	Chart.js	Lazy-loaded with the results screen only
Frontend build	Angular CLI + npm	Dev server on port 4200
Frontend tests	Vitest	
Backend language	Java 21 (LTS)	Records for DTOs, switch expressions, virtual threads enabled
Backend framework	Spring Boot 3.5.11	Web, Data JPA, Security, Validation, Actuator
Security	Spring Security + JWT 0.12.6	Stateless; access token plus rotating refresh token
Persistence	Spring Data JPA / Hibernate 6	<code>open-in-view: false</code> , <code>ddl-auto: validate</code>
Database	PostgreSQL 17	JSONB for marking configuration; partial and expression indexes
Migrations	Flyway	Owns the schema; runs automatically on boot
API documentation	springdoc-openapi 2.8.6	OpenAPI 3; Swagger UI on <code>dev</code> only
Boilerplate	Lombok	Entities only; DTOs are records
Backend build	Maven (wrapper included)	Maven itself need not be installed
Backend tests	JUnit 5, Mockito, AssertJ	Unit tests need no database

4.2 Prerequisites for a developer machine

- **JDK 21** — `java -version` should report 21.x
- **Node.js 20+** and npm
- **PostgreSQL 17** running locally, or any reachable PostgreSQL 14+
- **Git**

Maven is not required — the repository ships the Maven wrapper (`./mvnw` , or `mvnw.cmd` on Windows).

4.3 Why these choices

The stack is deliberately conservative. A first release has to be cheap to run and easy to reason about, so the project avoids microservices, Kafka, Redis and Kubernetes entirely. One Spring Boot process and one PostgreSQL database serve the whole platform. PostgreSQL earns its place beyond storage: JSONB holds marking configuration without a migration per rule, partial unique indexes enforce invariants the application would otherwise have to police under concurrency, and a window function computes the entire leaderboard in a single pass.

5 System architecture

5.1 Shape of the system

```
Angular 22  (standalone, zoneless, signals)
  |
  |  HTTPS / REST, JWT bearer
  v
Spring Boot 3.5  (modular monolith, one deployable)
  |
  |-- auth      registration, login, rotating refresh tokens, recovery
  |-- user      accounts, roles, student roster
  |-- catalog   subjects and chapters
  |-- question  the LaTeX question bank
  |-- marking   marking schemes and evaluation strategies
  |-- exam      tests, publication, question selection
  |-- attempt   sitting a paper: snapshot, autosave, submit, evaluate
  |-- ranking   leaderboards and percentiles
  |-- analytics dashboards and question quality
  |
  v
PostgreSQL 17
```

Each module owns its own `controller / service / repository / entity / dto / mapper` layers. Controllers carry no business logic, and **no JPA entity is ever serialised to a client** — every response is a record DTO. The module boundaries are real seams along which services could be extracted later if that ever becomes necessary; it is not necessary now.

5.2 Where decisions live

Every business decision is server-side. The browser is a rendering layer: it displays the server countdown, submits selections, and repaints from the server acknowledgement. Nothing about scoring, timing, eligibility or ranking is computed in the client, and the client is never sent data it should not be able to see.

5.3 Request path for an authenticated call

```
Browser
|  Authorization: Bearer <accessToken>
v
CORS filter                origins from CORS_ALLOWED_ORIGINS
v
JwtAuthenticationFilter    validates signature, expiry and the token TYPE claim
                             populates SecurityContext with UserPrincipal
v
Authorization              path rules (/admin/** = ROLE_ADMIN) + @PreAuthorize
v
Controller                 @Valid on the request record -> 400 with fieldErrors
v
Service                    business rules, ownership checks, transactions
v
Repository / Hibernate    -> PostgreSQL
v
Mapper -> record DTO -> JSON  nulls omitted, entities never serialised
```

5.4 Background work

Job	Cadence	Purpose
Attempt expiry sweep	every 30 seconds	Finalises attempts whose deadline has passed, scoring the answers already saved, so an abandoned paper is still marked.
Refresh-token purge	daily, 03:30	Deletes refresh tokens that expired more than 30 days ago.
Ranking recomputation	on every evaluation	Recomputes the whole cohort for the test in one PostgreSQL window-function pass.

6 Project structure

6.1 Repository layout

```
MathStrokesTestPlatform/
  backend/                Spring Boot service
    pom.xml
    mvnw, mvnw.cmd        Maven wrapper
    src/main/java/com/mathstrokes/
      MathStrokesApplication.java
      auth/ user/ catalog/ question/ marking/
      exam/ attempt/ ranking/ analytics/
      common/              dto, enums, exception, domain, util
      config/              AppProperties, JwtProperties, Jackson, OpenAPI
      security/            SecurityConfig, jwt/, service/
      bootstrap/           runtime seeders (admin account, sample tests)
    src/main/resources/
      application.yml, application-dev.yml, application-prod.yml
      db/migration/        V1__initial_schema.sql, V2__seed_reference_data.sql,
                           V3__seed_sample_questions.sql
  frontend/              Angular application
    src/app/
      core/               auth/, config/, guards/, http/, models/
      features/
        auth/             login, register, forgot-password
        admin/            layout/, home/, questions/, data/
        student/          home/, tests/
        exam/             exam-runner, exam.service
        results/          result-page, review-page
      shared/             brand/, forms/, math/
  docs/                  API.md, ARCHITECTURE.md, DEPLOYMENT.md
  e2e/                   end-to-end checks against a running instance
  README.md
```

6.2 Backend module anatomy

Every module follows the same six-layer shape, so any module reads like any other:

controller/	HTTP surface only — routing, <code>@Valid</code> , status codes, OpenAPI annotations. No business logic.
service/	Business rules, transaction boundaries, ownership checks, orchestration.
repository/	Spring Data interfaces plus JPA Specifications for dynamic filtering.
entity/	JPA entities extending <code>BaseEntity</code> (id, createdAt, updatedAt). Never serialised.
dto/	Java records. Request records carry Bean Validation annotations; response records carry the wire shape.
mapper/	Entity to DTO translation. The only place the two representations meet.

6.3 Frontend anatomy

core/auth/	<code>AuthService</code> (HTTP), <code>AuthStore</code> (signals-based session state), <code>TokenStorage</code> .
core/http/	<code>authInterceptor</code> attaches the bearer token; <code>TokenRefreshCoordinator</code> serialises concurrent refreshes so one 401 cannot trigger a stampede.
core/guards/	<code>authGuard</code> , <code>roleGuard(...)</code> , <code>guestGuard</code> , <code>landingGuard</code> . Convenience only — the backend enforces authorisation for real.
core/models/	Enums and interfaces mirroring the backend contract, kept in lockstep by hand until the OpenAPI document is used for generation.
shared/math/	<code>latex-renderer</code> and the <code>MathContent</code> component — the single place KaTeX is invoked.
features/	One folder per feature, each lazily loaded. The exam runner pulls in KaTeX and the results screen Chart.js; neither belongs in the bundle a student downloads to reach the login screen.

7 Data model

Seventeen application tables, created by `v1__initial_schema.sql`. Every table carries `created_at` and `updated_at`.

7.1 Identity

Table	Columns and constraints
users	<code>id</code> , <code>full_name</code> , <code>phone_number</code> UNIQUE, <code>password_hash</code> , <code>security_question</code> , <code>security_answer_hash</code> , <code>enabled</code> , <code>last_login_at</code> . CHECK <code>phone_number ~ '^[0-9]{10,15}\$'</code> . Neither hash is mapped into any DTO.
roles	<code>id</code> , <code>name</code> UNIQUE, <code>description</code> . Exactly two rows: <code>ROLE_ADMIN</code> , <code>ROLE_STUDENT</code> .
user_roles	Join table, PK (<code>user_id</code> , <code>role_id</code>).
refresh_tokens	<code>token_id</code> UNIQUE, <code>token_hash</code> , <code>user_id</code> , <code>issued_at</code> , <code>expires_at</code> , <code>revoked_at</code> , <code>replaced_by</code> . Stored hashed so a database leak cannot be replayed.

7.2 Catalogue

Table	Columns and constraints
subjects	<code>name</code> UNIQUE, <code>code</code> UNIQUE, <code>description</code> , <code>active</code> , <code>display_order</code> .
chapters	<code>subject_id</code> FK, <code>name</code> , <code>description</code> , <code>active</code> , <code>display_order</code> . UNIQUE (<code>subject_id</code> , <code>name</code>).

7.3 The question bank

Column	Meaning
<code>subject_id</code> , <code>chapter_id</code>	FK. Subject is derived from the chapter on write, never sent by the client.
<code>exam_pattern</code>	CHECK IN (<code>'JEE_MAIN'</code> , <code>'JEE_ADVANCED'</code>)
<code>difficulty</code>	CHECK IN (<code>'EASY'</code> , <code>'MEDIUM'</code> , <code>'HARD'</code>)
<code>question_type</code>	CHECK IN (<code>'SINGLE_CORRECT'</code> , <code>'MULTIPLE_CORRECT'</code>)
<code>question_content</code>	TEXT NOT NULL — LaTeX source, never rendered HTML . CHECK not blank.
<code>solution_content</code>	TEXT, optional — LaTeX source of the worked solution.
<code>status</code>	CHECK IN (<code>'DRAFT'</code> , <code>'PUBLISHED'</code> , <code>'ARCHIVED'</code>), default <code>DRAFT</code> .
<code>marking_scheme_id</code>	Nullable override. NULL means resolve the active scheme for (pattern, type).
<code>created_by</code> , <code>updated_by</code>	FK <code>users</code> , ON DELETE SET NULL.
<code>published_at</code>	Set when the question is first published.
<code>version</code>	@Version — Hibernate optimistic lock <i>and</i> the human-visible revision number recorded on attempt snapshots.

```
question_options: question_id FK ON DELETE CASCADE, option_key VARCHAR(5), content TEXT,  
display_order, is_correct BOOLEAN, with UNIQUE (question_id, option_key).
```

THREE INDEPENDENT AXES

Exam pattern, difficulty and question type are orthogonal. A JEE Advanced question can be EASY, and a JEE Main question can be HARD. The input control a student sees — radio versus checkbox — follows question_type, never exam_pattern, because a JEE Advanced paper may contain single-correct questions.

7.4 Marking schemes

`marking_schemes`: `name` UNIQUE, `exam_pattern`, `question_type`, `configuration` JSONB, `active`. A **partial unique index** permits at most one active scheme per `(exam_pattern, question_type)`, so resolution is never ambiguous.

```
configuration = {
  "fullCorrectMarks": 4.00,
  "wrongMarks": -1.00,
  "unansweredMarks": 0.00,
  "partialCreditMode": "NONE" | "PER_CORRECT_OPTION",
  "marksPerCorrectOption": 0.00,
  "maxPartialMarks": 0.00,
  "noPartialCreditMarks": 0.00
}
```

7.5 Tests

Column	Meaning
<code>title</code> , <code>description</code>	Shown to students in the test list.
<code>subject_id</code> , <code>chapter_id</code>	The scope the paper is drawn from.
<code>exam_pattern</code>	Determines which questions are eligible and which schemes apply.
<code>duration_minutes</code>	Default 60. CHECK between 1 and 600.
<code>question_count</code>	Default 25. CHECK between 1 and 200.
<code>easy_count</code> / <code>medium_count</code> / <code>hard_count</code>	Difficulty blueprint. NULL means no constraint on that band.
<code>generation_mode</code>	FIXED_SET or RANDOM_PER_ATTEMPT.
<code>status</code>	DRAFT, PUBLISHED, CLOSED, ARCHIVED.
<code>ranking_enabled</code>	True only for FIXED_SET — only identical papers form a fair cohort.
<code>max_attempts_per_student</code>	Default 1. CHECK between 1 and 100.

`test_questions` materialises the paper for a `FIXED_SET` test, written once at publication: UNIQUE `(test_id, question_id)` and UNIQUE `(test_id, question_order)`.

7.6 Attempts and the snapshot

`test_attempts` holds the lifecycle (`started_at`, `expires_at`, `submitted_at`, `evaluated_at`) and, once evaluated, the whole result: `score`, `max_score`, `negative_marks`, the four outcome counts, `accuracy`, `attempt_rate`, `time_taken_seconds`, `rank_position`, `total_candidates` and `percentile`. A **partial unique index** on `(student_id, test_id)` WHERE `status = 'ACTIVE'` means the database itself guarantees one live attempt per test, so two concurrent start requests cannot both succeed.

THE SNAPSHOT — THE SINGLE MOST IMPORTANT DESIGN ELEMENT

`attempt_questions` stores `question_version`, the stem, the solution, the difficulty, the type, `marking_scheme_name`, `marking_config` JSONB and `max_marks`. `attempt_question_options` stores each option key, content, order and `is_correct` — the answer key as it stood at snapshot time. Evaluation reads **only** these tables. The FK back to `questions` exists for analytics and is never followed for content or correctness.

7.7 Answers and results

Table	Purpose
<code>student_answers</code>	One row per attempt question. Holds <code>answer_status</code> , <code>visited</code> , <code>marked_for_review</code> , <code>client_sequence</code> and <code>answered_at</code> . UNIQUE on <code>attempt_question_id</code> .
<code>student_answer_options</code>	The selections, normalised — a row per selected option, not a comma-separated string.
<code>question_attempt_results</code>	Per-question outcome: <code>result_status</code> , <code>marks_awarded</code> , <code>max_marks</code> , selected and correct option counts. UNIQUE on <code>attempt_question_id</code> — which is what makes evaluation idempotent at the database level.

7.8 Status vocabularies

Enum	Values
<code>QuestionStatus</code>	DRAFT, PUBLISHED, ARCHIVED
<code>TestStatus</code>	DRAFT, PUBLISHED, CLOSED, ARCHIVED
<code>AttemptStatus</code>	NOT_STARTED → ACTIVE → SUBMITTED AUTO_SUBMITTED → EVALUATED (terminal)
<code>AnswerStatus</code>	NOT_VISITED, NOT_ANSWERED, ANSWERED, MARKED_FOR_REVIEW, ANSWERED_AND_MARKED_FOR_REVIEW
<code>QuestionResultStatus</code>	CORRECT, PARTIALLY_CORRECT, INCORRECT, UNANSWERED

8 Adding a question — full walkthrough

This is the core administrator procedure. It is given twice: first as the screen-by-screen flow an administrator follows, then as the exact API contract behind it.

8.1 Before you start

- You must be signed in as `ROLE_ADMIN`. Signing in lands you on `/admin`.
- The chapter you want to file the question under must already exist and be active — see section 11.
- An **active marking scheme** must exist for the (exam pattern, question type) pair you intend to use, or publication will fail. The four seeded schemes cover every combination out of the box.

8.2 The screen flow

1 Open the question bank

From the admin shell, go to **Question bank** — route `/admin/questions`.

The grid lists questions newest-updated first, 20 to a page, showing chapter, exam pattern, difficulty, type, a preview of the stem, status, option count and revision. Filter by subject, chapter, exam pattern, difficulty, question type and status, or search free text across the stem and solution. **Reset** clears every filter and returns to page 1.

2 Start a new question

Click **New question** — route `/admin/questions/new`.

The editor opens with four blank options already laid out and labelled **A, B, C, D**, and with sensible defaults selected: exam pattern `JEE_MAIN`, difficulty `MEDIUM`, question type `SINGLE_CORRECT`.

3 Classify the question

Set the four classification fields at the top of the form:

Field	Values	Notes
Chapter	Any active chapter	Required. The subject is derived from it automatically — there is no separate subject field.
Exam pattern	<code>JEE_MAIN</code> , <code>JEE_ADVANCED</code>	Required. Selects which marking scheme applies.
Difficulty	<code>EASY</code> , <code>MEDIUM</code> , <code>HARD</code>	Required. Used by the test difficulty blueprint. Independent of exam pattern.
Question type	<code>SINGLE_CORRECT</code> , <code>MULTIPLE_CORRECT</code>	Required. Decides radio versus checkbox for the student, and which evaluation strategy scores it.

Switching the type to single-correct collapses any extra correct selections down to the first, so the form cannot sit in a state the server would reject.

4 Write the question stem

Type the stem into **Question content** as LaTeX and plain text. A live KaTeX preview renders beside the textarea as you type. Maximum 20,000 characters; required.

Solve for x :

$$x^2 - 5x + 6 = 0$$

Blank lines are meaningful paragraph breaks and are preserved verbatim. See section 10 for the full LaTeX reference.

5 Fill in the options

Four option rows are provided, labelled A to D. Each row takes LaTeX content — required, up to 5,000 characters — and has its own live preview. `displayOrder` is taken from the row position, so the A–D order stays consistent without your having to manage it.

Option labels must be a **single uppercase letter** and must be unique within the question. A question needs between **2 and 10** options.

6 Mark the correct answer

Tick the correctness control on each option that is part of the answer key.

- `SINGLE_CORRECT` — the form behaves as a radio group: selecting one option clears the others.
- `MULTIPLE_CORRECT` — any number may be ticked, but not all of them (see step 9).

7 Add the worked solution (optional but recommended)

Solution content takes LaTeX up to 20,000 characters, with its own live preview. It is shown to students only on the answer-review screen, after their attempt is finalised — never during a live paper.

Left empty, the field is omitted from the request rather than sent as an empty string.

8 Save as a draft

Click **Save**. The question is created with status `DRAFT` and you are navigated to its edit route, `/admin/questions/{id}`.

Draft rules are deliberately permissive so half-finished work can be saved. Only these are checked on save: content not blank, option labels unique and single uppercase letters, 2–10 options, length caps, and a markup guard (see the box below).

9 Publish

Click **Publish**. Publication applies the strict rules, and only a question that passes them can enter the bank that test generation draws from:

- **At least two options.**
- `SINGLE_CORRECT` **must have exactly one correct option** — the error names the count it actually found.
- `MULTIPLE_CORRECT` **must have at least one correct option, but not all of them** — an all-correct question is unanswerable.
- **Option labels must be unique.**
- **An active marking scheme must exist** for the (exam pattern, question type) pair. This is checked at publish time so it fails here rather than at a student first click.

On success the status becomes `PUBLISHED` and `published_at` is stamped. The question is now eligible for test generation.

THE CONTENT SAFETY GUARD

On every save the server rejects content containing `<script`, `<iframe`, `<object`, `<embed`, `javascript:` or an inline `on...=` event handler. The check is deliberately narrow: `$a < b$` is ordinary mathematics and is accepted. The client is the first line of defence — it escapes everything outside the maths delimiters and runs KaTeX with HTML trust disabled — and this is defence in depth behind it.

8.3 The API contract behind the form

POST /api/admin/questions · ROLE_ADMIN · 201 Created

```
{
  "chapterId": 3,                                // required - subject is derived from it
  "examPattern": "JEE_MAIN",                     // required - JEE_MAIN | JEE_ADVANCED
  "difficulty": "MEDIUM",                       // required - EASY | MEDIUM | HARD
  "questionType": "SINGLE_CORRECT",              // required - SINGLE_CORRECT |
MULTIPLE_CORRECT
  "questionContent": "Solve  $x^2-5x+6=0$ ",      // required, <= 20000 chars, LaTeX source
  "solutionContent": "...",                     // optional, <= 20000 chars
  "markingSchemeId": null,                      // optional override; null = resolve active
scheme
  "options": [                                   // required, 2..10 entries
    { "optionKey": "A", "content": " $x=2$  or  $x=3$ ", "displayOrder": 0, "isCorrect": true
  },
    { "optionKey": "B", "content": " $x=-2$  or  $x=-3$ ", "displayOrder": 1, "isCorrect": false
  },
    { "optionKey": "C", "content": " $x=1$  or  $x=6$ ", "displayOrder": 2, "isCorrect": false
  },
    { "optionKey": "D", "content": " $x=-1$  or  $x=-6$ ", "displayOrder": 3, "isCorrect": false
  }
  ]
}
```

Field-by-field validation

Field	Required	Rule and message
chapterId	yes	<code>@NotNull</code> — "Chapter is required". Must resolve to an existing chapter.
examPattern	yes	<code>@NotNull</code> — "Exam pattern is required".
difficulty	yes	<code>@NotNull</code> — "Difficulty is required".
questionType	yes	<code>@NotNull</code> — "Question type is required".
questionContent	yes	<code>@NotBlank</code> , <code>@Size(max=20000)</code> . Trimmed on write.
solutionContent	no	<code>@Size(max=20000)</code> .
markingSchemeId	no	When absent, the active scheme for (pattern, type) is resolved at evaluation time.
options	yes	<code>@NotEmpty</code> , <code>@Size(min=2, max=10)</code> — "A question must have between 2 and 10 options".
options[].optionKey	yes	<code>@Pattern("^[A-Z]\$")</code> — "Option label must be a single uppercase letter". Uppercased and de-duplicated on write.
options[].content	yes	<code>@NotBlank</code> , <code>@Size(max=5000)</code> . Trimmed on write.
options[].displayOrder	no	<code>@PositiveOrZero</code> . Defaults to the array index.
options[].isCorrect	—	Boolean, defaults false. This is the answer key.

VALIDATION IS TWO-TIERED

Save-time rules apply to drafts too and are permissive. **Publish-time** rules are strict, and are enforced both on `POST /{id}/publish` and on any update to an already-published question — so a published question cannot be edited into an invalid state while remaining published.

ERRORS BIND STRAIGHT TO THE FORM

Every validation failure reports `fieldErrors[].field` as the JSON property name of the request body — `questionContent` , `solutionContent` , `options` , `options[2].content` — so the Angular form binds a server message onto the exact input that caused it, with no translation layer.

Worked example — a JEE Advanced multiple-correct question

```
POST /api/admin/questions
Authorization: Bearer <adminAccessToken>
Content-Type: application/json

{
  "chapterId": 12,
  "examPattern": "JEE_ADVANCED",
  "difficulty": "HARD",
  "questionType": "MULTIPLE_CORRECT",
  "questionContent": "Which of the following are true for  $f(x)=x^3-3x$ ?",
  "solutionContent": " $f'(x)=3x^2-3=3(x-1)(x+1)$ , so ...",
  "options": [
    { "optionKey": "A", "content": " $f$  has a local maximum at  $x=-1$ ", "isCorrect": true },
    { "optionKey": "B", "content": " $f$  has a local minimum at  $x=1$ ", "isCorrect": true },
    { "optionKey": "C", "content": " $f$  is increasing on  $(-1,1)$ ", "isCorrect": false },
    { "optionKey": "D", "content": " $f$  is odd", "isCorrect": true }
  ]
}

201 Created -> { "id": 87, "status": "DRAFT", "version": 0, ... }

POST /api/admin/questions/87/publish
200 OK -> { "id": 87, "status": "PUBLISHED", "publishedAt": "...", "version": 1, ... }
```

Three of four options are correct, which passes: at least one, but not all.

9 Question lifecycle



State	Reached by	Meaning
DRAFT	Creation, or revert	Editable freely under permissive rules. Invisible to test generation.
PUBLISHED	POST /{id}/publish	Eligible for test generation. Still editable, but every edit must continue to satisfy the strict publish rules.
ARCHIVED	POST /{id}/archive	Withdrawn from future tests. Read-only — attempting to update it returns a business-rule error telling you to restore it to draft first. Existing attempts and analytics are unaffected.

9.1 Editing a published question

Editing a question that has already been used in a live examination is **allowed and safe**. Each attempt holds its own snapshot of the content, options and answer key, so no historical result can move. What changes is the `version` number, which increments on every persisted write and is recorded on any future attempt snapshot that uses the question.

`version` also serves as a Hibernate optimistic lock, so two administrators editing the same question cannot silently overwrite each other — the second write fails rather than winning by accident.

VERIFIED END TO END

An automated check rewrites a question stem, flips its answer key and archives it, then re-reads a student existing attempt: the score, the counts, the per-question marks, the stem and the key all come back byte-identical.

9.2 There is no delete

The API exposes no hard delete for questions. **Archive is the terminal state**, because attempts, per-question results and analytics all reference the row. Removing it would break referential integrity and destroy history; the foreign keys are declared `ON DELETE RESTRICT` to make that impossible rather than merely discouraged.

9.3 Lifecycle endpoints

Method	Path	Purpose
GET	/admin/questions	Paginated search. Filters: <code>subjectId</code> , <code>chapterId</code> , <code>examPattern</code> , <code>difficulty</code> , <code>questionType</code> , <code>status</code> , <code>search</code> . Default 20 per page, sorted by <code>updatedAt</code> descending; page size capped at 100.
GET	/admin/questions/{id}	Full question including options and the answer key.
POST	/admin/questions	Create as a draft. 201 .
PUT	/admin/questions/{id}	Update. Options are merged by label so keys stay stable.
POST	/admin/questions/{id}/publish	Apply strict rules and publish.
POST	/admin/questions/{id}/draft	Return a published question to draft.
POST	/admin/questions/{id}/archive	Withdraw from future tests.

ANSWER KEYS AND ROUTE SAFETY

Every response under `/admin/questions` includes `isCorrect` on each option. That is precisely why the entire path sits behind `ROLE_ADMIN` , and why the student-facing attempt DTO is a different type with no correctness field at all rather than the same type with a filter applied.

10 Writing questions in LaTeX

The teacher types LaTeX; the browser renders it with KaTeX; the source is what gets stored.

```
Admin types LaTeX -> KaTeX live preview -> saved as SOURCE in PostgreSQL
                                     |
                               KaTeX renders <- student fetches the source
```

Rendered HTML is never persisted. The stored value stays the single source of truth, cannot carry markup into a page, and can be re-rendered by a different renderer later without a migration.

10.1 Delimiters

Form	Use	Example
$\$ \dots \$$	Inline	If α and β are the roots...
$\$ \$ \dots \$ \$$	Displayed	$x^2 + 5x + 6 = 0$

These two forms only. Stems, every option and the solution all support LaTeX.

10.2 Layout

Content is multi-line and blank lines are meaningful paragraph breaks — they are preserved verbatim, so a stem can be laid out for readability.

Solve for x :

$$x^2 - 5x + 6 = 0$$

10.3 Common constructs

Source	Renders as
$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$	The quadratic formula, displayed
$\int_0^1 x^2 \, dx$	A definite integral
α, β, θ	Greek letters, inline
$\lim_{x \rightarrow 0} \frac{\sin x}{x}$	A limit, inline
$\sum_{k=1}^n k^2$	A summation, displayed
$-\frac{7}{3}$	A fraction inside an option

10.4 What is not allowed

Content is LaTeX and plain text, **never HTML**. The server refuses to save content containing `<script` , `<iframe` , `<object` , `<embed` , `javascript:` or an inline event handler. The check is narrow by design so ordinary mathematics is never caught by it — `$a < b$` saves without complaint.

10.5 Authoring advice

- **Watch the preview.** It is the same renderer the student will see. If it does not render for you, it will not render for them.
- **Keep options parallel.** Options that differ only in the value being tested read more fairly than options that differ in form.
- **Write the solution while the question is fresh.** It is the only thing a student has to learn from after the paper, and it costs far more to reconstruct later.
- **Derive the key, do not assume it.** The seeded sample questions were generated from known integer roots rather than typed by hand, precisely so every answer key agrees with its own worked solution.

11 Managing subjects and chapters

Subjects and chapters are the spine the whole platform hangs off: questions are filed under a chapter, tests are scoped to one, and analytics report by chapter. The student app reads them from the API rather than hardcoding them, so adding a chapter needs no release.

11.1 Endpoints

Method	Path	Purpose
GET	/admin/catalog/subjects	Every subject, including inactive ones
POST	/admin/catalog/subjects	Create a subject. 201 .
PUT	/admin/catalog/subjects/{id}	Update a subject
GET	/admin/catalog/chapters?subjectId=	Chapters, optionally filtered by subject
GET	/admin/catalog/chapters/{id}	One chapter
POST	/admin/catalog/chapters	Create a chapter. 201 .
PUT	/admin/catalog/chapters/{id}	Update a chapter
PATCH	/admin/catalog/chapters/{id}/active?active=	Activate or deactivate

11.2 Field rules

Field	Rule
Subject.name	Required, at most 100 characters, unique
Subject.code	Required, at most 20 characters, unique, <code>^[A-Z0-9_]+\$</code> — uppercase letters, digits and underscores only
Chapter.subjectId	Required
Chapter.name	Required, at most 150 characters, unique within its subject
description	Optional, at most 500 characters
displayOrder	Optional, zero or positive — controls ordering in every dropdown and list

DEACTIVATE RATHER THAN DELETE

Chapters carry an `active` flag and the foreign keys from `questions` and `tests` are `ON DELETE RESTRICT`. Deactivating hides a chapter from new authoring and from the student app while leaving every existing question and result intact.

12 Marking schemes

A marking scheme is the configuration that turns an answer into a mark. Schemes are data, not code, so a new marking rule needs no release and no migration.

12.1 Endpoints

Method	Path	Purpose
GET	/admin/markings-schemes	List all schemes
POST	/admin/markings-schemes	Create. 201 .
PUT	/admin/markings-schemes/{id}	Update
PATCH	/admin/markings-schemes/{id}/active? active=	Activate or deactivate

12.2 Configuration fields

fullCorrectMarks	Awarded for a perfect answer. Must be greater than zero.
wrongMarks	Awarded for an incorrect answer. Must be zero or negative.
unansweredMarks	Awarded when nothing was selected. Normally 0.
partialCreditMode	NONE or PER_CORRECT_OPTION .
marksPerCorrectOption	Per-option award under partial credit. Must be greater than 0 when partial credit is on.
maxPartialMarks	Cap on a partial award. Must be greater than 0 and strictly less than fullCorrectMarks .
noPartialCreditMarks	Awarded for a strict subset of the key when partial credit is off.

CONFIGURATIONS ARE VALIDATED ON WRITE

A partial award that could match or beat a perfect answer is rejected, as is a positive penalty. A broken scheme therefore cannot reach an attempt snapshot. A **partial unique index** also permits at most one *active* scheme per (exam pattern, question type), so resolution is never ambiguous.

12.3 The four seeded schemes

Pattern / type	Correct	Partial	Wrong	Blank
JEE Main — single correct	+4	—	−1	0
JEE Main — multiple correct	+4 exact	none	−1	0
JEE Advanced — single correct	+3	—	−1	0
JEE Advanced — multiple correct	+4 exact	+1 per correct option, capped at +3	−2	0

13 Creating and publishing tests

A test is a blueprint plus a lifecycle. It names a scope (subject, chapter, exam pattern), a shape (duration, question count, difficulty mix) and a generation mode, and it draws from the published question bank.

13.1 The blueprint

Field	Default	Notes
title / description	—	What the student sees in the test list.
subjectId / chapterId	—	The scope questions are drawn from.
examPattern	—	Filters eligible questions and selects the marking schemes.
durationMinutes	60	1–600.
questionCount	25	1–200.
easyCount / mediumCount / hardCount	null	Difficulty blueprint. <code>null</code> means no constraint on that band.
generationMode	FIXED_SET	See below.
maxAttemptsPerStudent	1	1–100.

13.2 Generation mode decides whether ranking is possible

Mode	Behaviour
<code>FIXED_SET</code>	The question set is materialised once, at publish time, into <code>test_questions</code> . Every student receives exactly the same paper. This is the only mode that yields a fair ranking cohort , so <code>rankingEnabled</code> is true.
<code>RANDOM_PER_ATTEMPT</code>	Each attempt draws its own set from the blueprint. Useful for practice. Ranking is disabled — two such attempts are different examinations and comparing them would be meaningless.

13.3 Lifecycle

```
[ DRAFT ] --publish--> [ PUBLISHED ] --close--> [ CLOSED ] --archive--> [ ARCHIVED ]
  editable                paper is pinned          no new attempts
                        students can start          board settles
```

Method	Path	Purpose
GET	/admin/tests, /admin/tests/{id}	List and fetch
POST	/admin/tests	Create a draft
PUT	/admin/tests/{id}	Update — only drafts are editable
POST	/admin/tests/{id}/publish	Publish; a fixed-set test has its paper pinned here
POST	/admin/tests/{id}/close	Stop new attempts
POST	/admin/tests/{id}/archive	Withdraw from the list

PUBLICATION CAN FAIL — DELIBERATELY

If the published bank cannot satisfy the blueprint (too few questions of a required difficulty in that chapter and pattern), publication returns `NOT_ENOUGH_QUESTIONS` with 409. Publish a few more questions, or relax the blueprint, and try again. Failing here is far better than failing at a student first click.

14 Students and analytics

Method	Path	Purpose
GET	/admin/students?search=	The roster, searchable
PATCH	/admin/students/{id}/enabled? enabled=	Enable or disable an account
GET	/admin/analytics/dashboard	Headline counters
GET	/admin/analytics/questions/hardest	Lowest measured accuracy first
GET	/admin/analytics/chapters	Average score and accuracy by chapter

The hardest-questions report is measured, not assumed: it is computed from `question_attempt_results`, which carries a dedicated index on `(question_id, result_status)` for exactly this purpose. A question whose difficulty was set to `EASY` but which nobody answers correctly will surface here, and that is the signal to revisit the question rather than the students.

15 The marking engine

Marks are computed on the server, from a configuration stored as JSON, by one strategy per question type. Nothing about scoring is hardcoded in a controller, and nothing is computed in the browser.

```
public interface EvaluationStrategy {
    QuestionType supportedType();
    AnswerEvaluation evaluate(Set<Long> correctOptionIds,
                           Set<Long> selectedOptionIds,
                           MarkingConfig config);
}
```

Implementations are discovered as Spring beans and indexed by the type they support, so adding a question type means adding one class and one marking-scheme row — the evaluation pipeline itself does not change.

15.1 Single correct

Situation	Result
The key is selected	<code>fullCorrectMarks</code> , <code>CORRECT</code>
Anything else, including more than one selection	<code>wrongMarks</code> , <code>INCORRECT</code>
Nothing selected	<code>unansweredMarks</code> , <code>UNANSWERED</code>

15.2 Multiple correct — the rule ladder, in order

Situation	Result
Nothing selected	<code>unansweredMarks</code> , <code>UNANSWERED</code>
Any option outside the key selected	<code>wrongMarks</code> , <code>INCORRECT</code>
The key selected exactly	<code>fullCorrectMarks</code> , <code>CORRECT</code>
A strict subset of the key, nothing wrong	partial credit, <code>PARTIALLY_CORRECT</code>

ORDER MATTERS

The wrong-selection check comes **before** partial credit. That single ordering is what makes the JEE Advanced scheme come out right: a student who picks two correct options and one wrong one scores -2 , not $+2$.

Under `PER_CORRECT_OPTION` , the award is `marksPerCorrectOption` $\times$ `matchedCount` , capped at `maxPartialMarks` . With the seeded JEE Advanced configuration this reproduces the official scheme exactly: $+4$ for the exact key, $+1$ per correct option capped at $+3$, -2 for any wrong selection, 0 for no attempt.

15.3 Reported metrics

```
accuracy    = correct / attempted × 100    (0 when nothing was attempted)
attemptRate = attempted / total    × 100
```

A partially correct answer counts as **attempted but not correct**, so accuracy reads as the share of attempts that were fully right.

15.4 Precision

Marks are `NUMERIC(8,2)` in the database and `BigDecimal` in Java, throughout. Floating point is never used for a mark. With partial credit in play, a paper is a long chain of additions of values like 1.00 and -2.00, and binary floating point would accumulate error that eventually shows up as a wrong total on somebody result page.

15.5 Adding a new question type

1. Add the constant to the `QuestionType` enum.
2. Implement an `EvaluationStrategy` bean for it.
3. Insert a `MarkingScheme` row for each exam pattern it applies to.
4. Extend `QuestionValidator` with its publish rule.

Nothing else changes. Only types that have a registered strategy are declared in the enum, so the API never advertises a type that would fail at runtime.

16 Ranking, percentile and metrics

16.1 The cohort

Students are compared only when they sat the **same** paper. A test whose questions are drawn per attempt is not ranked at all. Only `FIXED_SET` tests — whose questions are materialised once at publication — carry `rankingEnabled: true`.

16.2 One position per student

If a test allows several attempts, only a student **best** attempt is placed, so sitting a paper twice cannot occupy two positions on the board.

16.3 Ties

RANK semantics: 1, 2, 2, 4. Genuinely tied students share a position and the next rank skips accordingly. Ties break on, in order:

1. Total score
2. More correct answers
3. Fewer incorrect answers
4. Faster completion
5. Earlier submission

16.4 Percentile

```
percentile = 100 × (candidates scoring at or below you) / total candidates
```

This follows the NTA definition and is computed on **raw score alone**, so a tie-break on time does not move it. The topper is at 100.00.

16.5 Recalculation

The whole cohort is recomputed whenever an attempt is evaluated, in one PostgreSQL window-function pass. A student who finishes first therefore sees a real position immediately, and that position can move as later candidates finish — which is what "rank among everyone who sat this paper" has to mean while the paper is still open. Once a test is `CLOSED`, no new attempts can start and the board settles.

CLIENT GUIDANCE

Because a rank shown at submission is a snapshot, a client should re-fetch the result rather than cache the rank indefinitely.

17 The student journey

1	GET	/api/tests	browse published papers; each says whether it can be started and whether one is in flight
2	POST	/api/attempts	start OR resume; the server draws the paper, SNAPSHOTs every question, writes started_at and expires_at, returns all 25 questions at once
3	PUT	/api/attempts/{id}/answers	autosave, checked four ways
4	POST	/api/attempts/{id}/submit	manual, or automatic at expiry
5	GET	/api/attempts/{id}/result	score, rank, percentile, metrics
6	GET	/api/attempts/{id}/review	answer key and worked solutions

17.1 Every autosave is checked four ways

1. The attempt belongs to the caller.
2. It is still `ACTIVE`.
3. The **server** clock says it has not expired.
4. The question belongs to that attempt.

17.2 Behaviours that matter

Behaviour	How it works
Refreshing changes nothing	Calling <code>POST /attempts</code> while an attempt is live resumes it: same questions, same order, same deadline. It never redraws the paper or restarts the clock.
The clock is the server	Every attempt response carries <code>timing: { serverTime, startedAt, expiresAt, remainingSeconds, expired }</code> . <code>remainingSeconds</code> is the server own arithmetic; <code>serverTime</code> lets the client correct for clock drift instead of trusting the browser.
Abandoning still gets marked	The expiry sweep finalises attempts whose time ran out, scoring the answers already saved.
Nothing is lost to a bad connection	Each save carries a monotonic <code>clientSequence</code> , incremented once per attempt rather than per question. A write with an older sequence is discarded, <code>accepted: false</code> is returned, and the response carries the newer stored state so the client reconciles instead of guessing.
Submission is idempotent	An attempt already submitted, or auto-submitted by the sweep, returns its result with 200 rather than an error — so the auto-submit race never needs a conflict screen.
The key is absent, not hidden	The live-attempt DTO has no correctness field and no solution text. They exist only on the review type.

17.3 Palette states

Each question carries one of five states, persisted server-side so the palette survives a refresh:

`NOT_VISITED` , `NOT_ANSWERED` , `ANSWERED` , `MARKED_FOR_REVIEW` , `ANSWERED_AND_MARKED_FOR_REVIEW` .

The refreshed 25-entry palette and the server clock come back with *every* save, so the navigator and the countdown update from the same response — no second call.

18 Authentication and security

Phone number and password. There is no email anywhere in the system.

18.1 Endpoints

Method	Path	Purpose
GET	/auth/security-questions	The canonical list, as <code>[{ id, text }]</code> . Submit the text at registration, not the id.
POST	/auth/student/register	Always creates a student. 201 .
POST	/auth/login	<code>{ phoneNumber, password }</code>
POST	/auth/refresh	Rotates both tokens
POST	/auth/logout	Revokes a refresh token. Idempotent.
POST	/auth/forgot-password/initiate	Returns only the security question
POST	/auth/forgot-password/verify	Answer it, receive a single-use reset token
POST	/auth/reset-password	Set the new password

18.2 Registration rules

Field	Rule
fullName	Required, 2–120 characters
phoneNumber	Required, <code>^[0-9]{10,15}\$</code> — 10 to 15 digits, no spaces or symbols. Unique; also enforced by a database CHECK.
password	Required, 8–72 characters, must contain an uppercase letter, a lowercase letter and a digit
confirmPassword	Required, must match
securityQuestion	Required, at most 255 characters
securityAnswer	Required, 2–200 characters. Matched case- and whitespace-insensitively.

18.3 Tokens

```
{
  "accessToken": "eyJ...",
  "refreshToken": "eyJ...",
  "tokenType": "Bearer",
  "expiresInSeconds": 900,
  "user": { "id": 3, "fullName": "...", "phoneNumber": "...",
    "roles": ["ROLE_STUDENT"], "enabled": true, "createdAt": "..." }
}
```

- The **access token** is a short-lived JWT — 15 minutes by default — sent as `Authorization: Bearer`.
- The **refresh token** is persisted as a SHA-256 digest with constant-time comparison, and is **rotated on every use**: presenting it revokes it and issues a replacement. Default lifetime 7 days.
- Presenting an already-rotated refresh token is treated as a stolen credential being replayed and **revokes every session for that account**.
- Every token carries a **type claim**, so an access token cannot be replayed as a refresh token or as a password-reset authorisation.
- Refresh returns a whole new pair — replace both values wholesale, never merge.

18.4 Password storage

Passwords **and security answers** are hashed with BCrypt at strength 12, so a database leak exposes neither. Neither hash is mapped into any DTO, so neither can be serialised into a response by accident.

18.5 Anti-enumeration

- Login cannot distinguish a wrong password from an unknown account — both return `AUTHENTICATION_FAILED`. Render one neutral message.
- Recovery cannot be short-circuited: knowing a phone number returns nothing but the question. Only a correct answer mints a reset token.
- The reset token authorises exactly **one** password change, and this is enforced without storing any server-side state: the token carries a fingerprint of the password hash it was minted against. Completing a reset changes the hash, the fingerprint stops matching, and a replayed token is refused even inside its ten-minute lifetime. Applying it also revokes every existing session for the account.
- Login, the security-answer challenge and recovery initiation are rate limited per identifier; **registration is throttled per client address**.
- Paginated endpoints cap page size at 100, so a single request cannot be used to drain a table.

18.6 Transport and hardening

- Stateless sessions; CSRF disabled because there are no cookies to forge against.
- CORS restricted to `CORS_ALLOWED_ORIGINS`, with `Authorization` exposed.
- Stack traces and framework messages are never included in error responses.
- `rejectedValue` is withheld for `password`, `confirmPassword`, `newPassword`, `currentPassword`, `securityAnswer`, `resetToken`, `refreshToken` and `accessToken` — so a submitted password is never written into a response body or a network log.
- Swagger and the OpenAPI document are disabled on the `prod` profile by default.

19 API reference

Base URL `http://localhost:8080/api` (context path `/api`). Generated OpenAPI at `/api/v3/api-docs`, Swagger UI at `/api/swagger-ui.html` — both on the `dev` profile only. Every route except `/auth/**`, `/actuator/health` and the docs requires `Authorization: Bearer`.

19.1 Catalogue and profile

Method	Path	Notes
GET	<code>/catalog/subjects</code>	Active subjects — any authenticated user
GET	<code>/catalog/subjects/{id}/chapters</code>	Active chapters. Do not hardcode these.
GET	<code>/profile</code>	The signed-in account, resolved from the token

19.2 Student — sitting a test

Method	Path	Notes
GET	<code>/tests</code>	<code>?subjectId=&chapterId=&examPattern=</code> . No question data.
POST	<code>/attempts</code>	<code>{ testId }</code> — starts or resumes . Returns the whole paper.
GET	<code>/attempts/active</code>	The attempt in flight, or 204 if there is none
GET	<code>/attempts/{id}</code>	Reload an attempt in progress
PUT	<code>/attempts/{id}/answers</code>	Autosave. Send the complete selection, never a delta.
PUT	<code>/attempts/{id}/questions/{qid}/visited</code>	Record that a question was seen
POST	<code>/attempts/{id}/submit</code>	Idempotent
GET	<code>/attempts/{id}/result</code>	409 while the attempt is live
GET	<code>/attempts/{id}/review</code>	A bare array , not an envelope. 409 while live.
GET	<code>/attempts/history</code>	Paginated
GET	<code>/me/performance</code>	Dashboard summary
GET	<code>/rankings/tests/{testId}</code>	Leaderboard

19.3 Admin

All under `/admin/**`, all requiring `ROLE_ADMIN`.

Path	Operations
/admin/catalog/subjects	GET, POST, PUT /{id}
/admin/catalog/chapters	GET, GET /{id}, POST, PUT /{id}, PATCH /{id}/active
/admin/questions	GET (paginated, filtered), POST, GET /{id}, PUT /{id}, POST /{id}/publish · /draft · /archive
/admin/marking-schemes	GET, POST, PUT /{id}, PATCH /{id}/active
/admin/tests	GET, POST, GET /{id}, PUT /{id} (drafts only), POST /{id}/publish · /close · /archive
/admin/students	GET ?search= , PATCH /{id}/enabled
/admin/analytics	GET /dashboard, /questions/hardest, /chapters

19.4 Paging envelope

```
{ "content": [], "page": 0, "size": 20, "totalElements": 0,
  "totalPages": 0, "first": true, "last": true }
```

Paginated endpoints accept `page`, `size` and `sort`.

20 Error handling contract

Every failure returns the same envelope:

```
{
  "timestamp": "2026-08-30T00:30:43.285Z",
  "status": 400,
  "error": "VALIDATION_ERROR",
  "message": "Validation failed",
  "path": "/api/auth/student/register",
  "fieldErrors": [
    { "field": "phoneNumber",
      "message": "Phone number must be 10 to 15 digits, with no spaces or symbols" }
  ]
}
```


error	Status	Meaning
VALIDATION_ERROR	400	Bad request data; see <code>fieldErrors</code>
AUTHENTICATION_FAILED	401	Missing or bad credentials
TOKEN_EXPIRED	401	Access token expired — refresh and retry
TOKEN_INVALID	401	Token unusable — sign in again
ACCESS_DENIED	403	Authenticated, but not allowed
RESOURCE_NOT_FOUND	404	No such resource
DUPLICATE_RESOURCE	409	Conflicts with existing data
BUSINESS_RULE_VIOLATION	409	A rule refused the operation
ATTEMPT_EXPIRED	409	Answer write after the deadline
ATTEMPT_ALREADY_FINALISED	409	Answer write to a submitted attempt
NOT_ENOUGH_QUESTIONS	409	The bank cannot satisfy the blueprint
RATE_LIMITED	429	Too many attempts; wait and retry
INTERNAL_ERROR	500	Unexpected; nothing leaks

TWO RULES FOR EVERY CLIENT

Branch on `error`, never on `message` — `error` is a stable enum name, `message` is human copy that may be reworded. **Nulls are omitted, not sent as `null`** — the application serialises with non-null inclusion, so model optional fields as `field?: T`. Commonly absent: `activeAttemptId`, `unavailableReason`, `markingSchemeId`, `fieldErrors`, and every result field before evaluation.

21 Installation runbook

1 Clone

```
git clone https://github.com/vivekkumarq/MathStrokes.git
cd MathStrokes
```

2 Create the database

As the `postgres` superuser:

```
CREATE ROLE mathstrokes LOGIN PASSWORD 'choose-a-password';
CREATE DATABASE mathstrokes OWNER mathstrokes;
```

On Windows, `psql` lives at `C:\Program Files\PostgreSQL\17\bin\psql.exe`.

3 Configure the environment

```
cp .env.example .env
```

Set at minimum `DATABASE_PASSWORD`, a strong `JWT_SECRET`, and the three `SEED_ADMIN_*` values. `.env` is gitignored and must never be committed.

```
openssl rand -base64 48          # generate a secret
```

The backend reads configuration from the environment, not from `.env` directly — export it first:

```
# bash / git-bash
set -a; . ../.env; set +a

# PowerShell
Get-Content ..\.env | Where-Object { $_ -match '^[A-Z]' } | ForEach-Object {
    $k,$v = $_ -split '=',2; [Environment]::SetEnvironmentVariable($k, $v.Trim(''))
}
```

4 Run the backend

```
cd backend
./mvnw spring-boot:run          # Windows: mvnw.cmd spring-boot:run
```

Flyway creates the schema and loads the reference data and sample questions on first start. The API comes up on `http://localhost:8080/api`. There is no separate migrate command.

5 Run the frontend

```
cd frontend
npm ci
npm start
```

Open `http://localhost:4200`.

6 Sign in and verify

Sign in with the seeded admin phone number and password. You should land on `/admin`. Open **Question bank** — the seeded questions should be listed, 20 to a page, with `totalElements` in the region of 1,522. Add one of your own by following section 8.

22 Environment variables

Variable	Required	Default	Notes
DATABASE_URL	yes	jdbc:postgresql://localhost:5432/mathstrokes	JDBC URL
DATABASE_USERNAME	yes	mathstrokes	
DATABASE_PASSWORD	yes	—	
JWT_SECRET	yes	—	At least 32 characters. The app refuses to start without it.
JWT_ACCESS_TOKEN_EXPIRATION	no	PT15M	ISO-8601 duration
JWT_REFRESH_TOKEN_EXPIRATION	no	P7D	
JWT_PASSWORD_RESET_TOKEN_EXPIRATION	no	PT10M	
CORS_ALLOWED_ORIGINS	yes in prod	http://localhost:4200	Comma separated
SPRING_PROFILES_ACTIVE	no	dev	dev or prod
PORT	no	8080	Context path is always /api
SEED_ENABLED	no	true	Set false in production once seeded
SEED_ADMIN_PHONE SEED_ADMIN_PASSWORD SEED_ADMIN_SECURITY_ANSWER	first run	—	All three are needed to create the first admin
SEED_ADMIN_NAME SEED_ADMIN_SECURITY_QUESTION	no	sensible defaults	
RATE_LIMIT_ENABLED RATE_LIMIT_MAX_ATTEMPTS RATE_LIMIT_WINDOW_SECONDS	no	true / 10 / 300	Per-identifier throttle
SWAGGER_ENABLED	no	false in prod	Enable only for a deliberate, temporary investigation
EXPIRY_SWEEP_INTERVAL_MS	no	30000	Attempt expiry sweep cadence
LOG_LEVEL_APP	no	INFO	DEBUG on the dev profile

NO CREDENTIALS IN THE REPOSITORY

The seeder refuses to invent a default admin password. If the three `SEED_ADMIN_*` values are missing it logs a warning and creates nothing, because a known default password on a deployed instance is an open door.

23 Database and migrations

Flyway owns the schema. Hibernate is set to `validate` and will refuse to start if the mapping and the schema disagree — so a drift between the two is caught at boot rather than at the first query that touches the changed column.

Migration	Contents
V1__initial_schema.sql	17 tables, constraints and indexes
V2__seed_reference_data.sql	Roles, Mathematics, the first 17 chapters, 4 marking schemes. Idempotent.
V3__seed_sample_questions.sql	66 published LaTeX questions
V4__seed_full_question_bank.sql	The remaining 11 chapters and 1,456 further questions
V5__seed_chapter_tests.sql	56 published chapter papers — 28 chapters × 2 exam patterns

WHY THE BANK IS A MIGRATION, NOT A SCRIPT

A deployed instance builds its database from migrations alone. Content created by POSTing to a running server exists only on the machine that received the POSTs, so seeding through Flyway is the only way a deployed site has a question bank at all. The payload in `V4` is dollar-quoted JSON rather than ordinary SQL string literals: the content is LaTeX, dense with backslashes and `$$` display-maths delimiters, and escaping that into normal literals is a reliable way to corrupt a macro or terminate a string early.

Migrations are **never edited once applied** — add a new `V4__...` instead. To start over locally:

```
DROP DATABASE mathstrokes;  
CREATE DATABASE mathstrokes OWNER mathstrokes;
```

24 Seed data

On a fresh database the application creates:

- **One admin**, from the `SEED_ADMIN_*` environment variables.
- **Mathematics** and its **28 chapters** — 17 from `V2` (Relations and Functions, Complex Numbers, Quadratic Equations, Sequences and Series, Permutations and Combinations, Binomial Theorem, Matrices, Determinants, Limits, Continuity and Differentiability, Applications of Derivatives, Integrals, Differential Equations, Vectors, Three Dimensional Geometry, Probability, Statistics) and 11 more from `V4`.
- **Four marking schemes**, one per (exam pattern, question type) pair.
- **Over 1,500 questions** with LaTeX stems, options and worked solutions, spanning all 28 chapters — precisely 1,522, being 66 from `V3` and 1,456 from `V4`, and the migrations set **every one of them to PUBLISHED**: there is not a single `DRAFT` or `ARCHIVED` row in the seed data.

- **59 published tests**, each 25 questions over 60 minutes: the 56 chapter papers from `v5` (one per chapter per exam pattern) plus the demonstration tests written by the runtime bootstrap seeder.

The sample questions were generated from known integer roots rather than typed by hand, so every answer key agrees with its own worked solution. Both runtime seeders skip silently if an admin or any test already exists, so restarting the application never overwrites real data.

TREAT THE QUESTION COUNT AS A STARTING FIGURE, NOT A PROPERTY OF THE SYSTEM

1,522 describes a *freshly migrated* database and nothing else. A published count drifts the moment anybody authors, drafts or archives anything, so any running instance will report something different — a development machine that has exercised the archive path, for example, will show 1,521 published and 1,522 total. If you need a figure that stays true, quote the **28 chapters** and **59 tests**: those are fixed by migration and do not move with ordinary administrative work.

THE SEEDED PAPERS ALLOW FIVE ATTEMPTS, NOT ONE

Every `v5` paper is `FIXED_SET` — the 25 questions are drawn once and pinned, which is what makes the cohort comparable and is the only mode under which ranking is enabled. But `max_attempts_per_student` is **5** rather than 1, because these are demonstration papers meant to be re-sat while exploring the marking schemes. **A real examination should use 1.**

25 Deployment

25.1 Current status

LIVE, WITH THE API PENDING

The frontend is deployed and public at <https://mathstrokes.netlify.app> — a **provisional address**, see below. The **backend is not yet deployed**: the site loads, but it cannot authenticate, because `/api/*` proxies to a placeholder until a backend host exists. Everything behind a sign-in is therefore unreachable on the public site today.

Note that the URL still carries the former product name: the rename to *lota* did not move the Netlify site. Whether to rename it is an open question at the time of writing, because changing the site name changes the public address. **Do not treat this hostname as final** — verify it before printing it anywhere a reader cannot easily correct.

25.2 It is three pieces, not one

This is the part a reader is most likely to get wrong. **Netlify cannot run a JVM**, so the Angular bundle and the Spring Boot service are hosted separately and stitched together by a proxy rather than deployed as one unit.

```

Browser
|
v
Netlify                serves the built Angular bundle (static files)
|  /api/* -- proxied by netlify.toml
v
Backend container      Spring Boot, on a host that runs containers
|
v
Managed PostgreSQL    Flyway migrates on first boot

```

Component	Approach
Frontend	<code>npm run build</code> produces static files. Served by Netlify (any static host or CDN would do).
The proxy	<code>netlify.toml</code> forwards <code>/api/*</code> to the backend. Because the browser only ever calls its own origin, there is no CORS in production at all — the <code>CORS_ALLOWED_ORIGINS</code> setting matters for local development and for any client that bypasses the proxy.
Backend	A container built from <code>backend/Dockerfile</code> , deployable to any host that runs containers, including free tiers. Configured entirely through environment variables.
Database	Any managed PostgreSQL. Flyway migrates on first boot.

GOTCHA: EVERY URL RETURNS 401 BEHIND A "LOGIN REDIRECT" PAGE

If a healthy Netlify deploy answers **401** on every URL, including the site root, the cause is almost certainly **SSO site protection enabled on the Netlify account** — not anything in the build. Nothing in the deploy log hints at it and the build itself reports success. Fix it by setting `sso_login` and `account_sso_login` to `false`. This cost twenty minutes once; it is written down so it costs nobody twenty minutes again.

25.3 Production checklist

- `SPRING_PROFILES_ACTIVE=prod`
- A strong `JWT_SECRET` from a secret store, never from source control
- `CORS_ALLOWED_ORIGINS` set to the real frontend origin
- `SEED_ENABLED=false` once the first admin exists
- `SWAGGER_ENABLED` left off
- The seeded admin password changed

EXPECT ONE FORCED SIGN-OUT AT THE REBRAND DEPLOY

The browser storage keys moved from `mathstrokes.auth.*` to `iota.auth.*`. Existing sessions cannot be read under the new keys, so **every signed-in user is signed out exactly once** when that build goes live. It happens a single time, not on subsequent deploys, and no server-side data is affected — but it is worth announcing rather than letting students discover it mid-revision.

On the `prod` profile, Flyway clean is disabled, stack traces and binding errors are excluded from responses, and root logging drops to `WARN`.

26 Testing

```
cd backend
./mvnw test           # 56 unit tests, no database required
./mvnw verify -Pintegration # PostgreSQL-backed integration tests
./e2e/run-all.sh      # 105 assertions against a running instance
```

Area	What is asserted
Single-correct scoring	Correct, wrong, unanswered, and a multi-selection scored as wrong rather than correct
Multiple-correct scoring	Exact match, partial credit, the cap, a wrong selection overriding partial credit, a superset of the key
Marking configuration	A partial award that could match a perfect answer is rejected; a positive penalty is rejected
Score aggregation	Totals, counts, negative marks, accuracy with nothing attempted, a negative total, time capped at the duration
Idempotent evaluation	An evaluated attempt is never rescored; existing results are never rewritten
Attempt lifecycle	An evaluated attempt is terminal and cannot be reopened
Palette state	All five states, including answered-and-marked-for-review
Publish rules	Exactly one key for single correct; at least one but not all for multiple correct; no duplicate labels
Content safety	<code>\$a < b\$</code> accepted; <code><script></code> , event handlers and <code>javascript:</code> refused
Security answers	Case- and whitespace-insensitive matching

Verified end to end against a running instance and PostgreSQL:

- A 25-question paper scored 55/100 for 15 correct, 5 wrong and 5 blank, with accuracy 75.00 and attempt rate 80.00 — matching an independently computed expectation.
- JEE Advanced partial marking scored 35 for 10 exact, 5 partial and 5 wrong.
- Refreshing resumed the same attempt, same questions, same order, clock not reset.
- A live attempt payload contained no correctness field and no solution text anywhere.
- A stale autosave carrying an older sequence was rejected and did not overwrite newer work.
- Another student was refused read, write and submit with 403.
- Submitting three times returned the same score every time.
- Expiry rejected further answers, the sweep auto-submitted and scored the saved work, and a late manual submit returned the result rather than a conflict.
- A teacher rewrote a stem, flipped its key and archived the question; the existing score, counts, per-question marks, stem and key were byte-identical afterwards.
- Four students on one test produced ranks 1, 2, 3, 4 with percentiles 100.00, 75.00, 75.00, 25.00, and a student who was rank 1 of 1 correctly became rank 2 of 4 as others finished.

27 Architecture decisions

Twelve decisions are recorded in full, with rationale and cost, in `docs/ARCHITECTURE.md`. In summary:

#	Decision	Why
1	A modular monolith, not microservices	One deployable is cheap to run and easy to reason about; the module seams remain available if extraction is ever needed.
2	Attempts carry an immutable snapshot	The only way to guarantee a historical result cannot move when a question is edited.
3	The server owns the clock	A client clock is attacker-controlled; the countdown is a display, not a source of truth.
4	Ranking compares only identical papers	Two randomly drawn papers are different examinations; comparing them is meaningless.
5	Marking is configuration plus a strategy	A new rule needs a row, not a release; a new type needs one class.
6	Money-like arithmetic uses BigDecimal	Partial marking accumulates error under floating point and eventually shows a wrong total.
7	Refresh tokens rotated and stored hashed	A leak cannot be replayed; a replayed token is detected and revokes the account sessions.
8	Ownership is derived, never accepted	No endpoint takes a student id, so no endpoint can be talked into acting as someone else.
9	The answer key is structurally absent	A different type cannot leak a field it does not have; a filter can be forgotten.
10	Idempotent submission	The auto-submit race needs no conflict screen. The first implementation deadlocked; the fix is documented.
11	Flyway owns the schema	Hibernate validates against it and refuses to start on drift.
12	Known limitations recorded honestly	See section 28.

28 Limitations and roadmap

28.1 Known limitations

Limitation	Impact and workaround
The OpenAPI document marks nothing as required	Java records carry no nullability metadata. Do not infer optionality from the generated spec — use the omit-null rule and the documented field tables instead.
Nulls are omitted, not sent	Optional fields are absent from responses. Any generated client must model them as optional or it will misread a valid payload.
Rate limiting is in-process and per-instance	Honest for a single-node deployment; behind a load balancer it would need a shared store.
No bulk question import	Questions are created one at a time via the API, or seeded through a Flyway migration.
No image upload on questions	Mathematics is expressed as LaTeX, so there is nothing to store. A question needing a diagram cannot currently be authored.
Only two question types	<code>SINGLE_CORRECT</code> and <code>MULTIPLE_CORRECT</code> . Only implemented types are declared, so the enum never advertises something that would fail at runtime.

28.2 Roadmap, in rough priority order

- **More question types** — `NUMERICAL`, `INTEGER`, `MATCHING`, `ASSERTION_REASON`, `COMPREHENSION`. The extension point is real: enum constant, one strategy bean, one marking-scheme row.
- **More subjects** — Physics and Chemistry need no schema change; `subjects` and `chapters` are already generic and the client reads them from the API.
- **OTP recovery** — the recovery flow is three separate steps precisely so an OTP challenge can replace the security answer without touching the reset step.
- **Required-field metadata in the OpenAPI document** — `@Schema(requiredMode = REQUIRED)` across the response DTOs.
- **Distributed rate limiting** — a shared store behind a load balancer.
- **Question versioning as first-class history** — attempts already snapshot everything they need, so a teacher-facing revision history would be purely additive.
- **Sub-chapters, bulk question import, and scheduled test windows.**

29 Glossary

Term	Meaning
Attempt	One student sitting of one test. Owns its own snapshot, clock and result.
Snapshot	The copy of question content, options, answer key and marking configuration taken onto an attempt when it starts. Evaluation reads only this.
Exam pattern	<code>JEE_MAIN</code> or <code>JEE_ADVANCED</code> . Selects the marking scheme. Independent of difficulty.
Question type	<code>SINGLE_CORRECT</code> or <code>MULTIPLE_CORRECT</code> . Decides radio versus checkbox and which evaluation strategy scores the answer.
Marking scheme	The JSONB configuration mapping outcomes to marks. At most one active per (pattern, type).
Partial credit	Marks for a strict subset of a multiple-correct key with no wrong selections. JEE Advanced only, in the seeded data.
Blueprint	A test difficulty mix — <code>easyCount</code> , <code>mediumCount</code> , <code>hardCount</code> .
Fixed set	A test whose questions are materialised once at publication so every student sits the same paper. The only rankable mode.
Palette	The 25-square navigator showing each question answer state.
Client sequence	A monotonic counter incremented once per attempt, used to discard out-of-order autosaves.
Sweep	The scheduled job that finalises attempts whose deadline has passed.
Percentile	NTA definition — the share of candidates scoring at or below you, computed on raw score alone.